CodeDNA: un protocollo di comunicazione in-source per agenti AI di coding (v0.9)

Fabrizio Corpora

Ricerca indipendente · *fabrizio.corpora@gmail.com*

Zenodo preprint · Aprile 2026 · Aggiornamento di doi:10.5281/zenodo.19158336 (v0.7, Marzo 2026)

Sommario

Gli agenti LLM che modificano autonomamente codebase di produzione soffrono di *cecità contestuale*: modificano una funzione senza consapevolezza delle dipendenze tra file, producendo bug silenziosi e cascate di errori. Le soluzioni esistenti — RAG, file di regole a livello di progetto (CLAUDE.md, AGENTS.md), o iniezione di contesto long-context — impongono latenza di recupero, sovraccarico di manutenzione o costi di infrastruttura. Un'indagine recente (ETH Zurigo, 2026) dimostra che i file di contesto generati da LLM possono *degradare* il tasso di successo del compito del 3% e aumentare i costi di inferenza di oltre il 20%. Presentiamo **CodeDNA**, uno standard di annotazione in-source che incorpora grafi di dipendenza, contratti pubblici, vincoli per gli agenti e convenzioni di naming semantico direttamente nei file sorgente. Il nome richiama la sua analogia biologica: come il DNA codifica il progetto completo dell'organismo in ogni cellula, CodeDNA garantisce che ogni frammento di codice porti sufficiente contesto architetturale perché un agente agisca correttamente in isolamento. Valutiamo CodeDNA v0.9 con una **valutazione SWE-bench multi-modello estesa** su 3 LLM (Gemini 2.5 Flash, DeepSeek Chat, Gemini 2.5 Pro) — **CodeDNA migliora la file localization F1 su tutti e tre i modelli**, con il risultato più forte e statisticamente robusto su **DeepSeek Chat**: **+17.1 punti percentuali** di guadagno medio F1 su **10 task Django** (50.9% → 68.0%, Wilcoxon esatto **p=0.0009765625**, vittorie/sconfitte/pareggi = 10/0/0). 6 dei 10 task DeepSeek sono stati **replicati indipendentemente** da un contributore esterno su una macchina separata, rafforzando la riproducibilità. Gemini 2.5 Flash mostra +13pp (60% → 72%, p=0.040) su 5 task e Gemini 2.5 Pro mostra +9pp (60% → 69%). Il protocollo v0.9 introduce tre nuovi campi — `related:` (link semantici cross-cutting), `wiki:` (puntatore opt-in a contesto curato più approfondito) e `message:` (layer conversazionale inter-agente) — insieme a un layer di tooling che trasforma le annotazioni in un vault di conoscenza navigabile compatibile con Obsidian. CodeDNA non richiede build step, sistemi esterni né latenza di recupero.

1. Introduzione

L'adozione di agenti LLM nei flussi di lavoro di produzione ha esposto una classe di fallimento che i benchmark su singolo file non colgono sistematicamente: la *cecità contestuale cross-file*. Quando un agente legge una funzione in isolamento — perché opera in modalità sliding-window o perché il contratto rilevante si trova in un altro modulo — non può osservare i vincoli architetturali che governano il comportamento corretto.

SWE-bench [1], il principale benchmark multi-file del settore, ha dimostrato nel 2024 che i modelli allo stato dell'arte riuscivano a risolvere solo l'1,96% delle issue reali di GitHub. La modalità di fallimento dominante non era la qualità del ragionamento, ma il *costo di navigazione*: trovare i file giusti richiedeva troppe interazioni con gli strumenti. Lavori successivi confermano il pattern: LoCoBench-Agent [2] (2025) misura l'efficienza nell'uso degli strumenti in flussi di lavoro long-context e rileva una sistematica sovraesplorazione; PRDBench [3] (2025) riferisce che gli agenti spendono la maggior parte del loro budget token in navigazione piuttosto che nella riparazione.

Le soluzioni attuali condividono una debolezza strutturale: *il contesto vive fuori dal codice*. CLAUDE.md e CursorRules sono file a livello di sessione invisibili agli agenti in modalità sliding-window. I sistemi RAG [6] introducono latenza di recupero e richiedono infrastrutture di embedding. Lo studio ETH Zurigo 2026 [4] ha riscontrato che i file di contesto generati da LLM (AGENTS.md) peggiorano il successo del compito del 3% e aumentano i costi di oltre il 20%, mentre i file scritti da umani offrono solo un guadagno del 4%.

Proponiamo un approccio complementare: incorporare il contesto *dentro il codice*, a ogni livello, in modo che qualsiasi frammento letto da un agente sia già autosufficiente.

Contributi:

(1) Il **CodeDNA Communication Protocol v0.9**: un protocollo di comunicazione in-source a quattro livelli (Level 0–3), language-agnostic, che estende v0.7 con tre nuovi cam-

pi — `related:` (link semantici cross-cutting che bypassano il grafo degli import), `wiki:` (puntatore opt-in a markdown curato più approfondito del terse header), e `message:` (layer conversazionale inter-agente per osservazioni non ancora abbastanza certe da diventare `rules:`).

(2) Un'analogia biologica che motiva il principio di ridondanza olografica dello standard.

(3) Una **valutazione multi-modello SWE-bench** su 3 LLM con una condizione DeepSeek estesa (10 task, 3 esecuzioni ciascuno), che dimostra miglioramenti ampiamente benefici nella file localization F1.

(4) Evidenza di **+17.1pp di guadagno medio F1** su DeepSeek Chat attraverso 10 task (Wilcoxon esatto p=0.0009765625, vittorie/sconfitte/pareggi 10/0/0), **+13pp** su Gemini 2.5 Flash (p=0.040), e **+9pp** su Gemini 2.5 Pro — con 6 dei 10 task DeepSeek **replicati indipendentemente** da un contributore esterno su una macchina separata.

(5) Un **layer di tooling** (`codedna wiki bootstrap` e `codedna wiki sync`) che converte le annotazioni CodeDNA in un vault di conoscenza navigabile compatibile con Obsidian e in una wiki narrativa di progetto in stile Karpathy — trasformando le annotazioni in-source in un grafo browsabile dall'umano.

(6) Rilascio open-source di spec, codice benchmark e file di integrazione AI.

2. Lavori Correlati

2.1 Benchmark Multi-File per Agenti

SWE-bench [1] valuta gli LLM su 2.294 issue GitHub reali su 12 repository Python, stabilendo che il coordinamento multi-file è il collo di bottiglia principale. **LoCoBench-Agent** [2] (novembre 2025) offre il parallelo metodologico più prossimo al nostro lavoro: valuta agenti in scenari di ingegneria del software long-context con 8.000 task e misura esplicitamente l'*efficienza nell'uso degli strumenti* — la stessa metrica di CodeDNA. **PRDBench** [3] (ottobre 2025, AAMAS 2026) usa 50 progetti Python reali con Product Requirement Documents e Agent-as-a-Judge, confermando che gli agenti sovraesplorano quando la struttura del codebase è opaca. **HumanEval** [5] misura la sintesi di singole funzioni da docstring: il suo *pass@k* è ortogonale alla navigazione cross-file, non competitivo.

2.2 Gestione del Contesto per Agenti LLM

RAG [6] recupera chunk rilevanti da un vector store a tempo di inferenza. Applicato al codice, la qualità del recupero dipende dalla similarità degli embedding, che può non allinearsi alla rilevanza architetturale. I **file di regole** (CLAUDE.md [10], CursorRules, AGENTS.md) impongono convenzioni a livello di progetto ma sono invisibili agli agenti in modalità sliding-window. Lo studio ETH Zurigo [4] fornisce la prova più forte contro l'approccio a file esterni. **MemGPT** [7] mantiene una memoria compressa tra i turni, ma una cronologia di letture precedenti non dice a un agente cosa leggere *dopo*. Gli approcci basati su knowledge graph (Tongyi Lingma [11]) usano la traversata del grafo per esporre le relazioni cross-modulo, ma richiedono infrastrutture esterne. **Il differenziatore chiave di CodeDNA**: il grafo di navigazione è co-locato con il codice stesso, non richiede sistemi di recupero e sopravvive in qualsiasi finestra sliding.

2.3 Standard di Annotazione e Documentazione

Gli standard di documentazione (docstring, JSDoc, Doxygen, Literate Programming [8]) si concentrano sulla leggibilità umana e non codificano semantica di navigazione cross-file né vincoli specifici per gli agenti. I sistemi di annotazione dei tipi (PEP 484, TypeScript) codificano contratti di tipo ma non architetturali. CodeDNA si distingue progettando le annotazioni come un *protocollo di navigazione machine-readable* che punta ai fail mode degli agenti LLM, non ai lettori umani.

3. Il Protocollo di Comunicazione CodeDNA v0.9

3.1 Analogia Biologica

Il nome *CodeDNA* cattura il principio di design dello standard attraverso un'analogia con la biologia molecolare. Il DNA biologico codifica il progetto completo dell'organismo in *ogni cellula*: ogni nucleo cellulare contiene il genoma completo, non solo le istruzioni per la funzione specializzata della cellula stessa. Questa ridondanza significa che ogni cellula — indipendentemente dalla sua posizione o contesto — può essere letta dalla giusta macchinaria e fornire informazioni architetturali sull'intero organismo.

CodeDNA applica lo stesso principio al software: ogni file sorgente codifica non solo la propria logica, ma anche la propria *posizione nell'organismo architetturale* — da cosa dipende, chi dipende da esso, quali contratti deve rispettare. Il protocollo comprende quattro livelli (0–3): il Livello 0 è il manifest di progetto (`.codedna`), che opera a scala di repository. I Livelli 1–3 operano a scala di file, funzione e variabile, e rispecchiano tre scale di codifica biologica:

Tabella A. Analogia DNA biologico ↔ CodeDNA

Concetto biologico	Equivalente CodeDNA
Genoma completo in ogni cellula	Module header L1: posizione architetturale completa in ogni file
Gene = unità funzionale locale	Docstring di funzione L2: contratto locale + dipendenze cross-file
Codone = unità minima di codifica	Commento inline L3: 1 riga di contesto nel punto di rischio
Ridondanza olografica	Qualsiasi estratto di 10 righe porta contesto sufficiente per navigare
Nessuna consultazione esterna	Zero latenza di recupero; il contesto viaggia con il codice

La proprietà olografica — tagliare un ologramma a metà produce due immagini complete più piccole, non due metà — spiega la robustezza di CodeDNA alla sliding-window: anche quando un LLM legge solo le righe 50–80 di un file, incontra docstring di funzione L2 e commenti inline L3 che riaffermano le regole architetturali critiche esattamente nel punto di rischio.

3.2 Livello 1 — Module Header

Una module docstring Python-nativa che deve essere il primo contenuto in ogni file. Codifica la posizione architetturale completa del file in circa 70 token.

```
"""analytics/revenue.py — Aggregazione ricavi mensili.

deps:     payments/models.py → get_invoices_for_period
          tenants/models.py → is_suspended
exports: monthly_revenue(year,month)->dict
used_by: api/reports.py → revenue_route
tables:   invoices(tenant_id, amount_cents, status)
          tenants(suspended_at, deleted_at)
rules:      get_invoices_for_period() restituisce TUTTI i tenant,
          NESSUN filtro sospensione → DEVE chiamare is_suspended()
          PRIMA di aggregare i ricavi
"""
```

L'uso del formato triple-quote nativo di Python (invece di un blocco di commenti personalizzato) massimizza la comprensione degli LLM: i modelli addestrati su miliardi di file Python applicano il pattern-matching esistente alla struttura della docstring, invece di analizzare un protocollo non familiare.

Il campo `rules` codifica vincoli rigidi che si applicano a ogni modifica nel file. A differenza delle annotazioni inline (a livello di funzione), `rules` si applica globalmente, agendo come il genoma della cellula: disponibile ovunque nel file senza richiedere al lettore di navigare verso una funzione specifica.

3.3 Livello 2 — Annotazioni Sliding-Window

Gli agenti spesso estraggono 50–100 righe dal mezzo di un file, bypassando il module header. Il Livello 2 incorpora cues di navigazione a livello di funzione attraverso due sotto-livelli.

Livello 2a — Docstring di funzione (stile Google):

```
def monthly_revenue(year: int, month: int) -> dict:
    """Aggrega le fatture pagate nei totali mensili.

    Dipende: payments.models.get_invoices_for_period
             — restituisce TUTTE le fatture, NESSUN filtro.
    Regole:  DEVE filtrare is_suspended() PRIMA di sommare.
    """
```

Livello 2b — Commento inline al sito di chiamata (ultima linea di difesa):

```
    # include tenant sospesi — filtrare sotto
    raw = get_invoices_for_period(year, month)
    # ZONA BUG: fatture sospesi incluse nel totale
    total = sum(i['amount_cents'] for i in raw)
```

Il commento al sito di chiamata garantisce che anche una finestra di 10 righe centrata sulla chiamata pericolosa porti il contesto critico. Questa è la codifica a livello di codone: l'unità più piccola che trasmette ancora significato architetturale.

3.4 Livello 3 — Naming Semantico

I nomi delle variabili codificano tipo, shape, dominio e origine. Formato: `<tipo>_<shape>_<dominio>_<origine>`.

```
# ❌ Standard — l'agente deve tracciare per capire
data  = get_users()
price = request.json["price"]

# ✅ CodeDNA — leggibile in qualsiasi finestra di contesto
list_dict_users_from_db  = get_users()
int_cents_price_from_req = request.json["price"]
```

3.5 Il campo `related:` (v0.9)

Il grafo `used_by:` cattura le dipendenze *strutturali* — chi importa chi. Molti bug reali, tuttavia, coinvolgono file che *condividono un pattern* senza alcun link di import: helper di codifica IDNA sparsi fra `validators`, `encoding`, `html` e `mail.utils`; gestione timezone duplicata fra moduli backend; idiom di equality check ripetuti in classi non correlate. Per queste *preoccupazioni cross-cutting*, nessuna catena di edge `used_by:` collega i file che l'agente deve modificare. Il protocollo v0.7 mostrava Δ≈0% su questa classe di task (si veda §6.5).

La versione 0.9 introduce il campo opzionale `related:` — un link *semantico* catturato accanto a quello strutturale:

```
"""mail/utils.py — helper email.

exports: class CachedDnsName | DNS_NAME
used_by: mail/message.py → DNS_NAME
related: core/validators.py — condivide logica
                              IDNA/punycode
         utils/encoding.py — utility di encoding
                             per domini non-ASCII
rules:   get_fqdn() ritorna unicode raw — i cal-
ler
         devono gestire non-ASCII
"""
```

Empiricamente, sul bug Django #11532 (crash per dominio unicode, fix su 5 file senza alcun antenato di import condiviso): un'annotazione CodeDNA che usa solo `used_by:` trova 2 file su 5 (F1=40%, equivalente al baseline di controllo). Aggiungendo `related:` l'agente arriva a **5 file su 5** (F1=100%). Il link semantico chiude il gap cross-cutting identificato come limite strutturale di v0.7.

3.6 Il campo `wiki:` opt-in e il layer knowledge-vault (v0.9)

Una singola riga `rules:` non può trasportare contesto narrativo — storia delle decisioni, extension point, invarianti sottili che richiedono un paragrafo. Scaricare tutto nel docstring gonfia il module header che la sliding window deve attraversare. v0.9 aggiunge il campo opzionale `wiki:`, un puntatore a una pagina markdown curata sotto `docs/wiki/`:

```
"""cli.py — tool di annotazione CodeDNA.

exports: scan_file | run
used_by: tests/test_cli.py → FileInfo
wiki:    docs/wiki/cli.md       ← opt-in
rules:   ...
"""
```

La sparsità è il segnale. Il workflow dell'agente è: se `wiki:` è presente, leggere prima il markdown puntato; se assente, il docstring è sufficiente. Un agente precedente ha lasciato il puntatore solo quando ha deciso che l'header terse era insufficiente. Diversamente dall'approccio "compila una wiki per ogni file", questo preserva il principio "ogni frammento porta il tutto" al default economico e fa opt-in in contesto più profondo solo dove serve.

Il tooling complementare — `codedna wiki bootstrap` e `codedna wiki sync` — converte il grafo di annotazioni in due artefatti complementari. **Bootstrap** emette un vault Obsidian annidato (una pagina markdown per file, con struttura che rispecchia l'albero sorgente, e `[[wikilink]]` derivati da `used_by:` e `related:`) — un grafo browsabile dall'umano senza alcuno sforzo di annotazione aggiuntivo. **Sync** rigenera una singola wiki narrativa di progetto (`docs/codedna-wiki.md`) seguendo un template di sette sezioni (identità, topologia, workflow, hotspot e protocollo di refresh) — progettata per agenti che entrano a freddo in una repository. Entrambi sono deterministici e idempotenti; collegati a un post-commit hook restano aggiornati senza dipendere dal fatto che l'agente ricordi di aggiornarli.

3.7 Il canale inter-agente `message:` (v0.9)

Ogni entry `agent:` può trasportare un sub-field `message:` — un layer conversazionale per osservazioni non ancora abbastanza certe da diventare `rules:`, domande aperte o note soft di handoff per il prossimo agente. Il campo vive a due livelli: dentro i module header (Livello 1) per agenti che leggono l'intero file, e dentro i docstring di funzione (Livello 2) per agenti che operano in sliding-window mode e non vedono mai l'header.

```
agent:   claude-sonnet-4-6 | 2026-03-10 | impl X
         message: "edge case di arrotondamento
                  multi-valuta — non sono sicuro
                  debba diventare una rule"

agent:   gemini-2.5-pro | 2026-03-18 | aggiunto Y
         message: "@prev: confermato → promosso
                  a rules:"
```

Un `message:` è append-only — un agente successivo o lo *promuove* a `rules:` (con `@prev: promosso a rules:`) o lo *respinge* con una ragione esplicita (`@prev: non applicabile perché...`). Entrambi gli esiti preservano il trail di audit completo. Nell'esperimento multi-agente SaaS AgentHub (DeepSeek R1, 5 agenti, 83 minuti), `message:` è stato adottato spontaneamente in 54 dei 55 file annotati (**98.2%**) senza alcun reminder a metà sessione — gli agenti lo hanno usato per note di handoff, annotazioni di gap per funzione e propagazione di vincoli cross-file.

4. Design del Benchmark

Misuriamo l'*efficienza di navigazione* — quante interazioni con gli strumenti richiede un agente autonomo per localizzare correttamente la causa radice di un bug in un codebase multi-file.

4.1 Implementazione dell'Agente

Abbiamo implementato un agente autonomo con Gemini 2.5 Flash (temperatura 0, deterministico) con tre strumenti function-calling: `read_file(path)`, `list_files(directory)` e `grep(pattern, directory)`. Nessun contenuto di file viene pre-caricato. L'agente gira fino al completamento (max 30 turni). Metriche: **chiamate agli strumenti totali**, **read_file**, **grep**, **turni di conversazione**, e **found_correct** (booleano).

5. Risultati

5.1 Valutazione Multi-Modello SWE-bench

Per verificare se il beneficio di CodeDNA si generalizza a diverse architetture LLM, abbiamo valutato 3 modelli su 5 issue Django reali da SWE-bench. Ogni modello ha ricevuto strumenti identici (`read_file`, `list_files`, `grep`), lo

stesso system prompt e un massimo di 30 turni. Ogni condizione è stata ripetuta 5 volte per task a T=0.1 per ottenere stime medie stabili; la significatività statistica è valutata con il test di Wilcoxon signed-rank (unilaterale, H1: CodeDNA > Control) sulle 5 coppie di F1 medie per task. La metrica primaria è la **file localization F1**: la media armonica di precision (frazione di file letti presenti nel ground truth del patch) e recall (frazione di file ground truth effettivamente letti dall'agente). Questa metrica misura l'*accuratezza di navigazione*, non la correttezza del patch, isolando il collo di bottiglia specificamente affrontato da CodeDNA.

Tabella 3. *File localization F1 multi-modello, Control vs. CodeDNA. DeepSeek copre 10 task; Gemini Flash e Pro coprono 5 task ciascuno. 3–5 esecuzioni per task a T=0.1.*

Modello	Task	Ctrl F1	DNA F1	Δ F1	p-value	Task Vinti
DeepSeek Chat	10	50.9%	68.0%	**+17.1pp**	0.001**	10 / 10
Gemini 2.5 Flash	5	60%	72%	**+13pp**	0.040*	4 / 5
Gemini 2.5 Pro	5	60%	69%	**+9pp**	0.11	3 / 5

Risultato chiave: DeepSeek Chat (10 task, W+=55, N=10) raggiunge una forte significatività statistica a **p=0.0009765625** (Wilcoxon esatto, one-tailed), con CodeDNA che vince su tutti i 10 task — nessuna inversione, nessun pareggio. Gemini 2.5 Flash (5 task, W+=14, N=5) raggiunge p=0.040 (Δ=+13pp). Gemini 2.5 Pro (5 task, W+=12, N=5) mostra Δ=+9pp a p=0.11. Il task 13495 mantiene la sua anomalia model-dependent — Flash beneficia fortemente (+22pp) mentre Pro regredisce (−8pp) — ma sotto la condizione DeepSeek estesa il Δ su 13495 è ora positivo (+6.6pp), suggerendo che la lettura negativa precedente riflettesse rumore di campionamento su N piccolo piuttosto che una regressione strutturale.

5.1.1 Replica indipendente su DeepSeek Chat

Fra la sottomissione v0.7 e questa revisione, un contributore esterno (@fabioscialanga) ha eseguito indipendentemente 6 task DeepSeek aggiuntivi (13121, 15629, 16263, 11400, 11883, 11808) su una macchina separata, usando lo stesso runner, gli stessi prompt e le stesse definizioni di task dei 4 task originali. Le session trace raw sono state integrate nel repository (`labs/benchmark/runs/deepseek-chat/session_traces/`) e l'analisi combinata sui 10 task rigenerata con lo script di analisi del repository. Il risultato è un dataset unico e coerente con provenienza trasparente: ogni session trace riporta il commit git e il run manifest della macchina di origine. Tutti i 10 task favoriscono CodeDNA. Due task (11808 e 13121) esibiscono un pattern di stabilità notevole — la deviazione standard di F1 fra tre run CodeDNA è **0.00** (l'agente produce la stessa decisione di file-localization ogni volta), mentre la std di controllo è 0.20–0.25. Il vantaggio non è meramente una media più alta; è una riduzione dell'imprevedibilità run-to-run che conta per i deployment di produzione. La replica indipendente su hardware eterogeneo con protocollo identico è la prova più forte che la dimensione corrente del campione consente; aumenta in modo significativo la fiducia che l'effetto DeepSeek non sia un artefatto di una singola macchina o di un singolo rollout del loop di campionamento.

I risultati supportano un'ipotesi di lavoro: *i modelli meno capaci e meno costosi beneficiano maggiormente di CodeDNA*. Un modello frontier naviga codebase complessi bene per ragionamento generale. Un modello più economico senza guida strutturale si perde, entra in loop o si ferma presto. CodeDNA fornisce l'impalcatura che permette a un modello economico di avvicinarsi alla qualità di navigazione di uno più costoso. Questo rende CodeDNA economicamente attraente: *annotare una volta, usare modelli più economici con accuratezza comparabile*.

Nota: questa ipotesi è supportata dal risultato di Flash (p=0,040, statisticamente significativo), ma DeepSeek-Chat e Gemini 2.5 Pro mostrano Δ positivi ma non significativi (p=0,11). È necessario uno studio multi-modello più ampio per confermare o confutare questa ipotesi.

Analisi qualitativa per task. I cinque task coprono uno spettro di difficoltà strutturale. **Task 12508** (dbshell): il punto di ingresso è ovvio dal nome del file; la difficoltà è scoprire i quattro client backend — `used_by:` lo gestisce in modo affidabile (+9pp). **Task 11991** (INCLUDE): `schema.py` è trovabile via grep; `used_by:` enumera esplicitamente tutti e quattro i backend schema editor (+17pp). **Task 14480** (Q() XOR): richiede di seguire una catena multi-livello ORM→SQL→backend su 7 file — esattamente la navigazione che `used_by:` abilita (+17–18pp). **Task 13495** (Trunc tzinfo): il task con la maggiore divergenza tra modelli. Flash guadagna +22pp; DeepSeek e Pro regrediscono di 8–9pp. Seguire le catene `used_by:` verso i file backend sembra introdurre una trappola di precisione per i modelli meno capaci — un'anomalia strutturale in fase di indagine. **Task 11808** (`__eq__`): identificato in v0.7 come limite strutturale del protocollo — il punto di ingresso `models/base.py` (847 righe) è un file generico senza catena navigabile verso le cinque sottoclassi interessate. v0.9 affronta questa classe di bug tramite il campo `related:` (§3.5), che permette a un file di puntare ai suoi fratelli semantici fuori dal grafo degli import. Sotto la condizione DeepSeek estesa sui 10 task, 11808 mostra ora Δ=+25.5pp — ma questo miglioramento mescola l'effetto di `related:` con differenze di modello. Un'ablation pulita (con/senza il campo `related:` sullo stesso modello) resta un'indagine aperta.

5.2 Analisi del Costo di Retry

L'F1 di file-localization è tipicamente presentato come metrica di efficienza. Proponiamo di trattarlo anche come *proxy di correttezza*: un agente che legge meno della metà dei file ground-truth difficilmente produrrà una patch completa, costringendo l'utente a ripetere l'intera sessione. Definiamo un run *ad alto rischio* quando il recall per-run scende sotto il 50%.

Sui 131 run multi-esecuzione del baseline v0.7 (DeepSeek-Chat, Gemini 2.5 Flash, Gemini 2.5 Pro — 5 task ciascuno, 3–6 run per task):

Tabella 4. *Rischio di fix incompleto (recall < 50%) aggregato su tre modelli e cinque task SWE-bench.*

Condizione	Run ad alto rischio	Run totali	Tasso di rischio
Control	34	66	**52%**
CodeDNA	16	65	**25%**
Riduzione			**−27 pp (−52% relativo)**

Il caso più marcato del baseline v0.7 era DeepSeek-Chat sul task 11808 (10 file ground-truth): tutti e cinque i run di controllo hanno ottenuto un recall del 10–20% (1–2 file su 10); tutti e cinque i run CodeDNA hanno ottenuto recall del 30–60%. Ogni run di controllo avrebbe prodotto una patch incompleta. L'espansione v0.9 e l'introduzione di `related:` rafforzano ulteriormente il risultato — DeepSeek sul task 11808 nella condizione a 10 task mostra Δ=+25.5pp F1 con std=0.00 sui tre run CodeDNA. Gemini 2.5 Pro offre un dato complementare: CodeDNA ha ridotto le chiamate medie agli strumenti del 14,3% aumentando contemporaneamente l'F1 medio di 9 pp — meno chiamate, maggiore accuratezza.

Questi risultati supportano un approccio *accuracy-first* per i protocolli di annotazione: anche se CodeDNA fosse neutro in termini di tool call, dimezzare il tasso di navigazione incompleta giustificherebbe la sua adozione dimezzando il numero di sessioni che l'utente deve ripetere.

6. Discussione

6.1 Posizionamento rispetto ai Lavori Correlati

Tabella 4. *Mappatura Problema–Soluzione.*

Lavoro	Problema esposto	Risposta CodeDNA
SWE-bench [1]	Navigazione multi-file è il collo di bottiglia	Riduce il costo di navigazione via grafo embedded
LoCo-Bench [2]	Inefficienza nell'uso strumenti	Riduce le chiamate agli strumenti su task con catene di dipendenza (−14% su Gemini 2.5 Pro con +9pp F1)
PRD-Bench [3]	Sovraesplorazione su codebase opachi	Struttura trasparente elimina l'esplorazione cieca
ETH Zurigo [4]	File di contesto esterni degradano −3%, costi +20%	Contesto embedded viaggia con il codice
MemGPT [7]	LLM dimentica contesto tra i turni	Nessuna memoria necessaria: contesto è nel codice
RAG [6]	Latenza di recupero e infrastruttura embedding	Zero recupero: contesto è già al momento della lettura

6.2 Perché l'Annotazione In-Source Funziona

Il meccanismo rispecchia l'intuizione biologica: quando il contratto che governa una funzione è dichiarato *alla funzione*, gli agenti capaci non hanno bisogno di recuperarlo dalla sua definizione originale. Quando un campo `rules` propaga il contratto a valle, nel luogo esatto in cui è violato, l'agente può diagnosticare il problema correttamente senza risalire alla definizione originale.

La scoperta ETH Zurigo [4] che i file di contesto generati da LLM *degradano* le performance motiva precisamente questo approccio: se i file di contesto esterni causano sovraesplorazione, la soluzione non è file migliori ma contesto embedded che arriva *con il codice che l'agente avrebbe comunque letto*.

6.3 Razionale del Formato

La scelta di module docstring Python-native (invece di blocchi di commenti personalizzati) per il Livello 1 è stata validata empiricamente: i modelli addestrati su corpora Python applicano il pattern-matching esistente alla struttura `deps:` / `exports:` / `rules:`. Il formato personalizzato `# === CODEDNA:0.5 ===`, pur essendo parsabile dalla macchina, richiedeva al modello di processare un schema sintattico non familiare, producendo catene di ragionamento marginalmente più lunghe.

6.4 Overhead di Annotazione e il Framing Agentico

Un'obiezione comune agli standard di annotazione è il costo di scrittura e manutenzione. Per CodeDNA, questa obiezione deve essere valutata nel contesto corretto. CodeDNA è progettato per *workflow di code generation agentica* — ambienti in cui un agente LLM genera e modifica i file sorgente. In questo contesto, l'agente stesso produce i module header e le function docstring come sottoprodotto

naturale della generazione. Lo sviluppatore umano non annota 500 file manualmente: l'agente di coding lascia la propria memoria architetturale nel codice mentre scrive. Il costo marginale dell'annotazione si avvicina a zero.

Questo framing ridefinisce anche cosa *è* CodeDNA: non uno standard che chiede agli sviluppatori di scrivere più commenti, ma una convenzione che dà agli agenti un formato strutturato in cui registrare la propria comprensione architetturale locale. Lo standard trasforma il codice generato dall'agente da output passivo in un grafo di navigazione attivo e auto-descrittivo per l'agente successivo che lo leggerà.

Un'osservazione secondaria: i campi strutturati di CodeDNA (`deps:`, `rules:`, `Depends:`) non richiedono nuova sintassi — vivono dentro docstring Python-native. Molte codebase hanno già docstring; CodeDNA aggiunge *intenzione di navigazione* alla documentazione già esistente. La distinzione tra una docstring standard e una CodeDNA non è sintattica ma semantica: quest'ultima deve essere *machine-oriented e navigation-oriented*, non solo human-readable.

6.5 Limitazioni

Dimensione del campione. La valutazione v0.9 usa 3–5 run per task a T=0.1, abilitando il test Wilcoxon. DeepSeek copre N=10 task (p=0.001); Gemini Flash e Pro N=5 ciascuno. Tutti i task provengono da un singolo repository Django, il che limita la generalizzabilità. I lavori futuri dovrebbero estendere Flash e Pro a N=10, aggiungere un secondo repository e valutare su task SWE-bench completi con toolchain agentiche reali (Devin-style, Cursor, GPT-tools loop).

Sincronizzazione delle annotazioni. La sfida di manutenzione principale non è la *scrittura* delle annotazioni (in workflow agentici il costo è quasi zero) ma garantire che rimangano *consistenti* col codice nel tempo. Se un agente modifica una funzione senza aggiornare i campi `rules:` o `Depends:`, l'annotazione diventa fuorviante — potenzialmente peggio di nessuna annotazione. Questo sposta il problema di ricerca dal costo di produzione all'*enforcement della consistenza*: pre-commit hook, validazione CI e obblighi di ri-annotazione lato agente sono le mitigazioni corrette. Le future integrazioni dovrebbero trattare l'annotation drift come modalità di fallimento di primo livello.

Valutazione multi-modello. Tutti e 3 i modelli sono completati. I risultati sono consistenti tra i modelli — Δ positivo su 10/10 task DeepSeek, 4/5 Flash, 3/5 Pro — ma il set di task è ancora solo Django. L'anomalia del task 13495 emersa in v0.7 (Flash +22pp, Pro/DeepSeek negativi) resta parzialmente visibile su Gemini Pro (−8pp) ma è diventata positiva su DeepSeek sotto la condizione estesa a 10 task (+6.6pp), suggerendo che la lettura negativa precedente riflettesse rumore di campionamento su N piccolo piuttosto che una regressione strutturale. Le differenze di runtime cross-provider (implementazioni di function calling, orchestrazione degli strumenti) introducono variabilità non completamente controllata.

Complessità dei bug. I task SWE-bench valutati richiedono la comprensione di uno o due contratti cross-file. I bug che si estendono su cinque o più layer architetturali potrebbero amplificare ulteriormente il vantaggio — un'ipotesi che richiede una valutazione su scala più ampia per essere testata.

6.6 Lavori Futuri

Copertura estesa del benchmark. La valutazione v0.9 copre 20 coppie F1 per-modello-task attraverso 3 LLM (10 DeepSeek + 5 Flash + 5 Pro). Le direzioni prioritarie includono: estendere Flash e Pro a 10 task ciascuno per eguagliare la dimensione del campione DeepSeek, aggiungere una condizione placebo (un header che sembra CodeDNA ma popolato con annotazioni plausibili ma false) per isolare la *correttezza* delle annotazioni dalla mera presenza di testo strutturato nell'header, e metriche di correttezza end-to-end della patch che complementino la F1 di navigazione (che questo paper misura in isolamento per design).

Generazione LLM-assistita di `related:`. Il campo `related:` chiude il gap cross-cutting empiricamente (§3.5, §6.5), ma la popolazione oggi è manuale o rule-based. Un'estensione proposta usa un singolo passaggio LLM per file per suggerire link semantici dall'intero corpus — rilevando pattern condivisi fra file non correlati — con l'agente che mantiene autorità di veto.

Telemetria agente tramite git trailer. Git è già immutabile, append-only e diff-completo. Un'estensione proposta incorpora i metadati di sessione AI direttamente nei messaggi di commit usando *git trailer* — la stessa convenzione di `Co-Authored-By:`, riconosciuta nativamente da GitHub. I campi proposti: `AI-Agent:`, `AI-Provider:`, `AI-Session:`, `AI-Visited:` (file letti durante la sessione, che git non traccia nativamente), e `AI-Message:`. Combinati con il log `.codedna` e il campo `agent:` nel file, creano un'*architettura di audit a tre livelli* collegata da un `session_id` condiviso: git (diff e metadati autorevoli), `.codedna` (sommario di sessione navigabile per la pianificazione dell'agente), e `agent:` nel file (one-liner sicuro in modalità sliding-window).

Pattern cross-cutting a livello di progetto (`cross_cutting_patterns:`). Il campo `related:` (§3.5) affronta le preoccupazioni cross-cutting a livello di *file*: ciascun file punta individualmente ai suoi fratelli semantici. Una direzione complementare è un'estensione Level 0 che aggiunge

un campo `cross_cutting_patterns:` al manifest `.codedna`, popolato da un agente *post-fix* come conoscenza accumulata a livello di progetto. Sotto la condizione DeepSeek estesa, il task 11808 mostra ora Δ=+25.5pp — ma questo miglioramento confonde l'effetto di `related:` con differenze di modello, quindi un'ablation pulita resta aperta.

CodeDNA come corpus di addestramento auto-generante. La seguente sezione descrive una direzione futura speculativa che non ha ancora ricevuto validazione empirica. Una proprietà consequenziale del protocollo è che una codebase annotata con CodeDNA con utilizzo attivo degli agenti costituisce un *corpus di addestramento auto-generante* a tre livelli: (a) *SFT* — il campo `agent:` append-only accumula un log ordinato di decisioni di navigazione corrette; (b) *DPO/RLHF* — la git history con trailer AI produce coppie *(rejected, chosen)* naturali quando Agent B corregge Agent A e aggiorna `rules:`; (c) *PRM* — l'infrastruttura di tracciamento delle sessioni registra l'intera sequenza di tool call per sessione come traiettoria di ragionamento etichettata. Questi tre livelli si autopotenziano costituendo un *data flywheel*: ogni sessione agente che scrive o corregge un'annotazione CodeDNA migliora il segnale di addestramento disponibile a tutte le generazioni future di modelli — senza ulteriore etichettatura umana. La codebase è contemporaneamente l'ambiente, il protocollo e il dataset di addestramento.

Agenti di verifica. Il rischio di sincronizzazione delle annotazioni identificato in §6.5 motiva una classe di *agenti di verifica* che periodicamente confrontano le annotazioni con la codebase reale. Poiché le annotazioni sono strutturate e machine-readable, questa manutenzione è essa stessa automatizzabile: verifica della correttezza della navigazione (`used_by:`), compliance delle regole (`rules:`), accuratezza delle API pubbliche (`exports:`) e ciclo di vita delle osservazioni (`message:`).

7. Conclusione

Abbiamo presentato CodeDNA, un protocollo di comunicazione in-source a quattro livelli (Level 0–3) il cui design è radicato in un'analogia biologica: come il DNA codifica il progetto completo di un organismo in ogni cellula, CodeDNA garantisce che ogni file sorgente — e ogni funzione al suo interno — porti sufficiente contesto architetturale perché un agente LLM navighi correttamente senza lookup esterni.

La nostra valutazione SWE-bench multi-modello fornisce evidenza che il contesto architetturale in-source riduce il costo di navigazione per gli agenti LLM. Il risultato più forte è su **DeepSeek Chat**: **+17.1pp** di guadagno medio F1 attraverso **10 task** Django (50.9% → 68.0%, Wilcoxon esatto p=0.0009765625, 10/0/0) — con 6 dei 10 task replicati indipendentemente da un contributore esterno. Gemini 2.5 Flash mostra +13pp (60% → 72%, p=0.040) e Gemini 2.5 Pro +9pp (60% → 69%), con Δ positivo su tutti e 3 i modelli.

Il risultato valida l'ipotesi centrale: *quando i contratti cross-file sono dichiarati nel punto d'uso, gli agenti possono navigare più efficientemente senza risalire al contratto alla sua sorgente.* v0.9 introduce tre feature — `related:`, `wiki:` e `message:` — che affrontano le tre limitazioni più rilevanti di v0.7 (preoccupazioni cross-cutting, contesto narrativo oltre l'header, memoria inter-agente). Le direzioni aperte rimanenti includono: fine-tuning su corpus annotati con CodeDNA così che i modelli trattino i campi strutturati come segnali nativi di navigazione; un manifest `cross_cutting_patterns:` a livello di progetto che complementi il `related:` a livello di file; un benchmark longitudinale di `message:` attraverso molte sessioni sulla stessa codebase.

La specifica (v0.9), il codice del benchmark, le session trace raw (inclusi i run DeepSeek replicati indipendentemente) e i file di integrazione AI sono disponibili su:

github.com/Larens94/codedna

Riferimenti

[1] Jimenez, C., et al. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770.

[2] *LoCoBench-Agent: An Interactive Benchmark for LLM Agents in Long-Context Software Engineering.* arXiv:2511.13998, novembre 2025.

[3] *PRDBench: Automatically Benchmarking LLM Code Agents through Agent-Driven Annotation and Evaluation.* arXiv, ottobre 2025. Presentato ad AAMAS 2026.

[4] ETH Zurigo / Univ. Berna (2026). *Context File Provision in LLM Coding Agents: When More Context Is Less.* arXiv:2602.11988.

[5] Chen, M., et al. (2021). *Evaluating Large Language Models Trained on Code (HumanEval).* arXiv:2107.03374.

[6] Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS 2020. arXiv:2005.11401.

[7] Packer, C., et al. (2023). *MemGPT: Towards LLMs as Operating Systems.* arXiv:2310.08560.

[8] Knuth, D. E. (1984). Literate Programming. *The Computer Journal*, 27(2), 97–111.

[9] GitHub Copilot. (2024). *Custom instructions for GitHub Copilot.* docs.github.com.

[10] Anthropic. (2024). *Claude's memory: CLAUDE.md.* docs.anthropic.com.

[11] Tongyi Lingma Team (2025). *Repository-Level Code Understanding via Knowledge Graph Traversal.* Alibaba DAMO Academy preprint.

[12] Karpathy, A. (2025). *LLM-wiki: compilare una codebase in un grafo di conoscenza navigabile.* GitHub Gist. gist.github.com/karpathy/442a6bf555914893e9891c11519de94f.

[13] Corpora, F. (2026). *CodeDNA v0.7 — preprint originale.* Zenodo. doi:10.5281/zenodo.19158336.